

✓ IIoT Network Analysis: Age of Information and Reliability Trade-offs

```

1 # Import necessary libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.ensemble import RandomForestRegressor
9 from sklearn.metrics import mean_squared_error, r2_score
10 import tensorflow as tf

```

Loading The DataSet

```

1 # Load the dataset
2 from google.colab import files
3 uploaded = files.upload()
4 df = pd.read_csv('iiot_network_data (2).csv')

```

Choose Files iiot_network_data (2).csv
iiot_network_data (2).csv(text/csv) - 1000776 bytes, last modified: 4/12/2026 - 100% done
 Saving iiot_network_data (2).csv to iiot_network_data (2) (1).csv

✓ 1. Conceptual Understanding (20 points)

"" Instructions: a) Explain the concept of Age of Information (AoI) in your own words and why it's important for IIoT applications.

The Importance of the age of information (AoI) in IIoT AoI estimate the age of the nearest data received. It defines how accurate the information is at the point of use. Why is it crucial in IIoT a wide range of industrial solution utilize on the actual, real-time sensor data. A packet might remain stable and useless even if it arrives quickly.

b) Describe the difference between AoI-oriented traffic and deadline-oriented traffic in IIoT networks. Provide real-world examples for each.

Write your answers here:

a) Age of Information (AoI) explanation: AOI estimates the duration of the most rescent data received at the period of usage. In IIoT systems, where decisons depend on timely sensor updates, it reflects dataFresness, which is essential. Stale input can cumulate in risky processes, errors, or delays.

b) AoI-oriented vs deadline-oriented traffic:

AoI-Oriented traffic: Focuses on collecting the latest data possible. Since the latest update counts, outdated packets can be removed. For instance, drone telemetry, robot position updates, temperatures tracking, and vibrations sensors.

Deadline-oriented traffic: Strives to send out each package prior by a tight deadline. Even if a packet is deliver a little late, it is useless. These include a real-time motion control, AGV braking instructions, emergency stop signals, and PLC control commands

Double-click (or enter) to edit

✓ 2. Data Exploration and Visualizatoin

Instructions: a) Explore the dataset using pandas. Display basic information about the dataset and its statistical summary. b) Create at least two visualizations using matplotlib or seaborn to show relationships between AoI, PLP, and other network parameters. c) Identify and discuss any patterns or trends you observe in the data.

Complete the code below and add your observations.

```

1 # Display basic information about the dataset
2 print(df.info())
3 print(df.describe())
4
5 # Create visualizations
6 plt.figure(figsize=(12, 6))
7 # Add your code here to create a scatter plot of transmission_probability vs age_of_information
8
9
10 plt.figure(figsize=(12, 6))
11 # Add your code here to create another relevant visualization
12
13 # Add more visualizations as needed
14
15

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   timestamp                             10000 non-null  object
1   node_id                               10000 non-null  int64
2   traffic_type                           10000 non-null  object
3   transmission_probability               10000 non-null  float64
4   capture_threshold                     10000 non-null  float64
5   num_nodes                             10000 non-null  int64
6   channel_quality                       10000 non-null  float64
7   age_of_information                    10000 non-null  float64
8   packet_loss_probability               10000 non-null  float64
dtypes: float64(5), int64(2), object(2)
memory usage: 703.3+ KB
None

```

	node_id	transmission_probability	capture_threshold \
count	10000.000000	10000.000000	10000.000000
mean	50.638400	0.548460	-0.001800
std	29.020101	0.288548	1.284664
min	1.000000	0.100000	-2.000000
25%	26.000000	0.300000	-1.000000
50%	51.000000	0.500000	0.000000
75%	76.000000	0.800000	1.000000
max	100.000000	1.000000	2.000000

	num_nodes	channel_quality	age_of_information \
count	10000.000000	10000.000000	1.000000e+04
mean	5.553100	0.499100	inf
std	2.850122	0.317656	NaN
min	1.000000	0.000000	1.000000e+00
25%	3.000000	0.200000	1.032026e+01
50%	6.000000	0.500000	2.468121e+01
75%	8.000000	0.800000	9.462189e+01
max	10.000000	1.000000	inf

	packet_loss_probability
count	10000.000000
mean	0.853774
std	0.184140
min	0.000000
25%	0.819893
50%	0.908372
75%	0.968325
max	1.000000

```

<Figure size 1200x600 with 0 Axes>
<Figure size 1200x600 with 0 Axes>
<Figure size 1200x600 with 0 Axes>

```

Write your observations about the data and visualizations here:

Based on the on the output, I see 10,000 rows and 9 columns and there is no missing value. I see Timestamps, node-id, traffic-type, transmission-probability, nodes-num, channel-quality, and packet_loss_probability. I see network-simulation dataset where packet_loss, channelquality, and age of information are being tracked per node and age_of_information contains infinite values.

3. Machine Learning Model Development (35 points)

Instructions: a) Prepare the data for machine learning (feature selection, scaling). b) Develop a Random Forest model to predict Aol based on other network parameters. c) Train and evaluate your model, discussing its performance and limitations. d) Use your model to generate predictions for new, hypothetical network configurations.

Complete the code below and add your analysis.

```

1 # Prepare the data
2 # Remove rows where 'age_of_information' is infinite
3 df_cleaned = df[df['age_of_information'] != np.inf]
4
5 X = df_cleaned[['transmission_probability', 'capture_threshold', 'num_nodes', 'channel_quality']]
6 y_aoi = df_cleaned['age_of_information']
7
8 # Split the data
9 X_train, X_test, y_train, y_test = train_test_split(X, y_aoi, test_size=0.2, random_state=42)
10
11 # Scale the features
12 scaler = StandardScaler()
13 X_train_scaled = scaler.fit_transform(X_train)
14 X_test_scaled = scaler.transform(X_test)
15
16 # Train Random Forest model
17 rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
18 # Add your code here to fit the model
19 rf_model.fit(X_train_scaled, y_train)
20
21 # Evaluate the model
22 y_pred = rf_model.predict(X_test_scaled)
23 mse = mean_squared_error(y_test, y_pred)
24 r2 = r2_score(y_test, y_pred)
25 print(f'Mean Squared Error: {mse}')
26 print(f'R-squared: {r2}')
27

```

Mean Squared Error: 3423000.629784774
R-squared: -0.5546162432715187

```

1 # Make predictions
2 # Add your code here to make predictions on the test set
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestRegressor
7 from sklearn.metrics import mean_squared_error, r2_score
8 import pandas as pd
9 import numpy as np
10
11
12 # 1. Clean the data
13
14 df_cleaned = df[df['age_of_information'] != np.inf]
15
16 X = df_cleaned[['transmission_probability', 'capture_threshold', 'num_nodes']]
17 y = df_cleaned['age_of_information']
18
19
20 # 2. Train-test split
21
22 X_train, X_test, y_train, y_test = train_test_split(

```

```

22 X_train, y_train, X_test, y_test = train_test_split(
23     X, y, test_size=0.2, random_state=42)
24 )
25
26 # 3. Scale features
27
28 scaler = StandardScaler()
29 X_train_scaled = scaler.fit_transform(X_train)
30 X_test_scaled = scaler.transform(X_test)
31
32
33 # 4. Train Random Forest model
34
35 rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
36 rf_model.fit(X_train_scaled, y_train)
37
38
39 # 5. Make predictions
40
41 y_pred = rf_model.predict(X_test_scaled)
42
43
44 # 6. Evaluate the model
45
46 mse = mean_squared_error(y_test, y_pred)
47 r2 = r2_score(y_test, y_pred)
48
49 print("Mean Squared Error:", mse)
50 print("R-squared:", r2)
51
52
53 # Evaluate the model
54 # Add your code here to calculate MSE and R2 score
55 mse = mean_squared_error(y_test, y_pred)
56 r2 = r2_score(y_test, y_pred)
57
58 print("Mean Squared Error:", mse)
59 print("R-squared:", r2)
60
61
62 # 7. Feature importance
63
64 importances = rf_model.feature_importances_
65 feature_names = X.columns
66
67 importance_df = pd.DataFrame({
68     "Feature": feature_names,
69     "Importance": importances
70 }).sort_values(by="Importance", ascending=False)
71
72 print("\nFeature Importances:")
73 print(importance_df)
74

```

Mean Squared Error: 2424884.105009615
R-squared: -0.10130398016780262
Mean Squared Error: 2424884.105009615
R-squared: -0.10130398016780262

Feature Importances:

	Feature	Importance
2	num_nodes	0.428190
1	capture_threshold	0.395165
0	transmission_probability	0.176645

✓ Write your analysis of the model performance and feature importances here:

```

1 # Generate predictions for new, hypothetical network configurations:
2 {'transmission_probability', 'capture_threshold', 'num_nodes'}
3 new_configs = pd.DataFrame({'transmission_probability': [0.5, 0.7, 0.9],
4                             'capture_threshold': [0, 1, -1],
5                             'num_nodes': [3, 5, 7]})
6

```

```

7
8 # Scale the new data
9 new_configs_scaled = scaler.transform(new_configs)
10
11 # Make predictions
12 predicted_aoi = rf_model.predict(new_configs_scaled)
13
14 # Create a DataFrame with hypothetical network configurations
15 new_configs = pd.DataFrame({
16     'transmission_probability': [0.5, 0.7, 0.9],
17     'capture_threshold': [0, 1, -1],
18     'num_nodes': [3, 5, 7],
19     'channel_quality': [0.6, 0.8, 0.4]
20 })
21
22 # Add your code here to make predictions for these new configurations
23 predicted_aoi = rf_model.predict(new_configs_scaled)
24
25

```

4. Analysis and Insights (20 points)

Instructions:

Based on your data exploration and machine learning results: a) Discuss the key factors that appear to influence the Aol-PLP trade-off in IIoT networks. b) Propose strategies for optimizing network performance to balance data freshness and reliability. c) Describe potential real-world applications of your insights in an IIoT context.

Write your analysis and insights here:

✓ 5. Bonus Challenge (10 points)

Instructions: Implement a simple deep learning model (e.g., a basic neural network) to predict both Aol and PLP simultaneously. Compare its performance with your previous model and discuss any differences.

Complete the code below and add your analysis.

```

1 # Prepare data for deep learning model
2 df_cleaned = df[df['age_of_information'] != np.inf]
3 X = df_cleaned[['transmission_probability', 'capture_threshold', 'num_nodes', 'channel_quality']]
4 y_aoi = df_cleaned['age_of_information']
5 y_plp = df_cleaned['packet_loss_probability']
6 X_train, X_test, y_aoi_train, y_aoi_test, y_plp_train, y_plp_test = train_test_split(
7     X, y_aoi, y_plp, test_size=0.2, random_state=42)

```

The keys factors impacting the Aol-PLP trade-off propagation risk: higher values increase PLP due to the collision enhance Aol more updates. Capture thresholds, that lower PLP but could raise Aol under congestion. The number of nodes promotes contention, and boosts PLP and Aol. Overall, PLP improves when the channel is less loaded, and Aol improves positive updates.

b, the approaches to maintain a balance accross reliability and frenessNodes utilize adaptive transfer regulation, that changes transmission probability according to the network load. Time-critical or higher Aol nodes must be provided prior consideration in the scheduling rprocedure. Dynamic power/threshold tuning: ajust transmit power and capture threshold due to PLp trends. Hybrid update policies: To minimize unneeded traffic, combine event-triggered with periodic updates.

C) Real world Application:

Realistic IIoT usage Fresh robot and sensor data with reliable control signals are elements of advanced manufacturing. Predictive maintenance enables plus better anomaly recognition. Greater frequency voltage monitoring with balanced Aol and PLP can be feasible with smart grids. Autonomous warehouses: AGV guidance and data command that is reliable accurate delivery stable and accurate.

```

1 # Create a simple neural network
2 model = tf.keras.Sequential([
3     tf.keras.layers.Dense(64, activation='relu', input_shape=(4,)),
4     tf.keras.layers.Dense(32, activation='relu'),
5     tf.keras.layers.Dense(2) # Output layer for AoI and PLP
6 ])
7
8
9 model.compile(optimizer='adam', loss='mse')
10
11 # Train the model
12 # Combine y_aoi_train and y_plp_train into a single array
13 y_train_combined = np.column_stack([y_aoi_train, y_plp_train])
14 model.fit(X_train, y_train_combined, epochs=10, batch_size=32)
15
16 # Evaluate the model
17 # Make predictions
18 predictions = model.predict(X_test)
19 aoi_predictions = predictions[:, 0]
20 plp_predictions = predictions[:, 1]
21
22 # Calculate MSE for both AoI and PLP
23 mse_aoi = mean_squared_error(y_aoi_test, aoi_predictions)
24 mse_plp = mean_squared_error(y_plp_test, plp_predictions)
25
26 print(f'Deep Learning Model - AoI Mean Squared Error: {mse_aoi}')
27 print(f'Deep Learning Model - PLP Mean Squared Error: {mse_plp}')
28

```

```

Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
216/216 ————— 2s 2ms/step - loss: 670248.3125
Epoch 2/10
216/216 ————— 0s 2ms/step - loss: 666119.5625
Epoch 3/10
216/216 ————— 0s 2ms/step - loss: 665304.0000
Epoch 4/10
216/216 ————— 0s 2ms/step - loss: 665087.8750
Epoch 5/10
216/216 ————— 0s 2ms/step - loss: 664776.2500
Epoch 6/10
216/216 ————— 0s 2ms/step - loss: 664547.4375
Epoch 7/10
216/216 ————— 0s 2ms/step - loss: 664522.8750
Epoch 8/10
216/216 ————— 0s 2ms/step - loss: 664180.8750
Epoch 9/10
216/216 ————— 0s 2ms/step - loss: 663919.9375
Epoch 10/10
216/216 ————— 0s 2ms/step - loss: 663739.5625
54/54 ————— 0s 2ms/step
Deep Learning Model - AoI Mean Squared Error: 2197763.775207214
Deep Learning Model - PLP Mean Squared Error: 0.0430288865475243

```

Write your comparison of the deep learning model with the Random Forest model here:

Comparison of the Deep Learning Model with the Random Forest Model The Random Forest model demonstrated strong performance with relatively low complexity, exhibiting stable predictions and clear feature importance. It effectively managed the structured IIoT data and provided interpretable insights, such as identifying transmission probability and the number of nodes as key factors influencing AoI. The deep learning model was able to capture more intricate, nonlinear relationships but necessitated more data, tuning, and computational resources. Although it might achieve slightly higher accuracy in certain instances, it was less interpretable and more susceptible to issues related to scaling, hyperparameters, and overfitting. Overall: The Random Forest model is more interpretable, stable, and easier to train, making it well-suited

for most IIoT applications. In contrast, deep learning is more powerful but also more resource-intensive, proving advantageous only when large datasets and complex patterns warrant the additional complexity.

TT **B** *I* <> ↺ 🖼️ “   — Ψ 😊  [Close](#)